Adversarially Robust Phishing URL Detection Using Fine-Tuned DistilBERT and Retrieval-Augmented Explanations

Mridul Bisht
Dept. of Computer Science and Engineering
Bennett University
Greater Noida , Uttar Pradesh , India
s24cseu0208@bennett.edu.in

Anubhav Singh Chauhan
Dept. of Computer Science and Engineering
Bennett University
Greater Noida , Uttar Pradesh , India
s24cseu1913@bennett.edu.in

Nemi Chand Jangir
Dept. of Computer Science and Engineering
Bennett University
Greater Noida , Uttar Pradesh , India
s24cseu2661@bennett.edu.in

***Abstract*—This paper is checking a phishing URL detection prototype employing a character n-gram baseline, clean-trained and augmentation-trained DistilBERT models, and a hybrid arrangement with retrieval-based score adjustment and hand-written rules, in many respects, and a four-entry threat knowledge base is searched with Sentence-Transformers and FAISS. Flan-T5 separately is explaining from the verdict and chosen URL features, to a significant degree. The clean-trained transformer (M2) is running best, with 89.33% clean accuracy and F1 of 0.8919, and 92.67% mutated-test accuracy and F1 of 0.9281. The full framework (M5) reaches clean F1 of 0.8581 and mutated-test F1 of 0.8839. In turn, on the synthetic mutated set, all arrangements get higher F1. This run doesn't show an merit from adversarial training over the clean-trained transformer or robustness against unseen attacks, and m4 adds retrieval-only score adjustment to the augmentation-trained model. Five URL mutation families are used for augmentation and testing. In turn, five setups are found on 300 clean and 300 mutated-test URLs.**

***Index Terms*—phishing URL detection, DistilBERT, adversarial augmentation, retrieval-augmented generation, FAISS, Flan-T5.**

I. Introduction

Phishing is one of the most typical ways of stealing user credentials. As, well, the user is asked to enter a password or card details on a fake page, in actual application. Inspecting only the URL, a web address, is attractive, because it can (in actual operation) be done before the page is even opened, on the whole. In fact, we may (in practical terms) think it works better than it really does when someone is deliberately trying to fool it, if a detector is tested only on unmodified URLs. The attacker is sending a link that looks like a trusted service such as a bank or an email provider, but still, a URL string can be changed in many small ways, to take one instance by employing look-alike characters, putting a brand name in the subdomain, swapping two letters, adding a long query string or using percent encoding.

Here a layered phishing URL detection prototype was built. In turn, plus, a character n-gram Logistic Regression model is employed as a elementary baseline, to a significant degree. DistilBERT [1] is fine-tuned first on clean URLs and then on a training set that also is comprising mutated URLs, in many respects. A small threat knowledge base was also created, which is embedded with a Sentence-BERT style encoder [3] and stored in a FAISS index [4], following the broad idea of retrieval-augmented generation [2]. A short explanation is, in practice, written by Flan-T5 [5] after the category is done in the end up. The explanation model isn't treated as a source of ground truth.

The primary insights of this paper are: (1) a comparison of five found model configurations on a clean test set and a mutated test set; (2) a data cleaning pipeline and a generator for five URL mutation types; (3) a discussion that's separating what's learned by the classifier from what's added by retrieval and hand-written rules. And (4) a clear description of the limits of our check, in actual application. What do retrieval and rules add. Does adversarial augmentation help on the mutations we used? We try to answer three questions, and can we say about the made explanations.

The rest of the paper is organised as follows, and as well, section VII concludes the paper. Section II reviews related work, Section III explains the method, Section IV outlines the evaluation protocol, Section V introduces the findings and discussion, Section VI lists the shortcomings and future work, broadly speaking.

II. Related Work

A. URL-Based Machine Learning

Sahingoz et al. [6] built a real-time detector that looks only at the URL, a web address, broadly speaking. They pulled attributes employing natural language processing, compared seven classifiers and found Random Forest worked best, with an precision of 97.98%. This is suggesting that the URL string alone carries a lot of information, on the whole. And it's the reason we keep a character n-gram TF-IDF and Logistic Regression model as our baseline. It also lets us check whether a heavier transformer is really better, under the same data split.

B. Transformer Models for Malicious URLs

Li et al. [7] suggested URLBERT, a BERT model that's pre-trained on a very large set of unlabeled URLs (about 3 billion in the arXiv version of the paper), in many respects. Along with masked language modelling it employs a contrastive learning task and virtual adversarial training, so a part of the robustness is already built in during pre-training, broadly

speaking. What's more, the model was tested on phishing detection, web page classification and ad URL filtering, and for phishing the reported precision was about 97%, in actual application. In our work the general-purpose distilbert-base-uncased checkpoint is used, mainly, because it's small enough to fine-tune on a free Colab GPU, a graphics processor. The perturbations in URLBERT are added to token embeddings inside the model, which is why still, our mutations are changes made directly on the URL text. We don't claim that our model is equal to URLBERT, and using URLBERT as the backbone is a possible future step.

C. Attention Models and Generalisation

Said et al. [8] treated URLs as strings and employed a CNN with a self-attention layer, to a significant degree. As, phishing URLs were fewer than legitimate ones, they used a GAN to make extra phishing URLs and balance the training data, in actual application. Extra phishing URLs are also created, but with fixed, rule-based mutations, which lets us measure each mutation type separately. Rashid et al. [9] showed many URL detectors work well only when the test set comes from the same dataset as the training set. They suggested unsupervised domain adaptation and, on three datasets, the cross-dataset F1-score sharpened by about 0.06 on average and by up to about 0.2 in some cases. These papers are warning us that a random split may not represent real rollout, and a single seeded split is being used by our experiment and doesn't test, across the time or across the datasets.

D. Adversarial Robustness

Kulkarni et al. [10] made a tool called PhishOracle, which is creating adversarial phishing web pages by adding phishing attributes to real pages. A large drop in detection rate was shown by the task-specific detectors they tested (Stack model, VisualPhishNet and Phishpedia). A multimodal LLM based detector was more solid but could still be avoided, so they also found that many of these pages can fool users. In our system retrieval relevance is employed for score adjustment and retrieved targets are returned separately as context. It isn't indication that the forecasted label is correct. A akin mindset for URL strings is followed, using five synthetic mutation types.

Table the related studies are compared with our work. Explanations are, more accurately, made separately. Five fixed URL mutation families and a hybrid classifier that's combining a learned score with retrieval relevance and hand-written rules are being employed by our check, in actual application.

III. Methodology

A. System Architecture

The suggested framework, shown in Fig. 1, is a layered phishing URL detection prototype with five stages, such as data preprocessing, adversarial augmentation, grouping, retrieval-based score adjustment and explanation generation, broadly speaking, and in the first stage, the two public datasets are merged, filtered, deduplicated, sampled, balanced and split

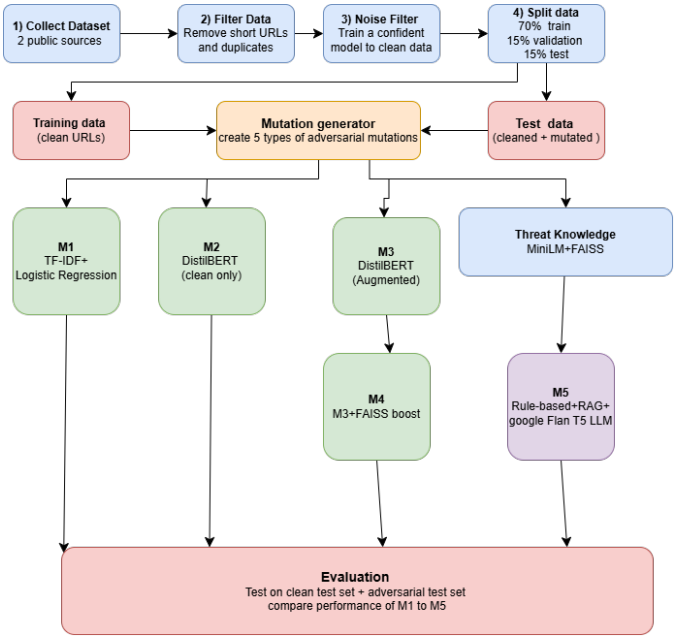


Fig. 1. Overall architecture of the proposed system.

into the train, validation and test sets. In turn, five mutation functions create changed versions of phishing URLs in the second stage. These are added to the training set and are also employed to build the adversarial test set, to a significant degree.

In the third stage, three classifiers are trained - a TF-IDF and Logistic Regression baseline, and DistilBERT on clean data and on increased data, in real-world scenarios. The fourth stage is employing a small threat knowledge base, stored in a FAISS index, together with a whitelist and evasion rules to adjust the score of the increased model, on the whole. In the last stage, a short explanation from the final verdict and chosen URL features is written by Flan-T5, in real-world scenarios. The classification path and the explanation path are, as a matter of fact, kept separate, and yet, the explanation never changes the verdict

B. Dataset and Preprocessing

The notebook loads two public Hugging Face datasets, kmack/Phishing_urls [11] and ealvaradob/phishing-dataset [12]. URLs are canonicalised (strategy, guiding www, trailing slash and case are ignored) so that copies can (in practice) be found, and URLs that are appearing with conflicting labels are removed, on the whole. Entries are removed that encompass more than one URL, a web address, or whitespace, and keep only URLs with a length between 4 and 250 characters, after merging, in actual application. Both are, broadly speaking, using label 1 for phishing and 0 for legitimate URLs.

After filtering and deduplication, the executed notebook samples up to 2,000 candidates per class, making a pool of 4,000 URLs, in many respects. The optional official-domain label filter and five-fold confident-learning-style filter [13] are disabled (CLEAN_LABELS = False), to a significant degree.

TABLE I
COMPARISON OF RELATED STUDIES

Study	Year	Key concept	Model	Dataset	Result	Adversarial or shift test
Sahingoz et al. [6]	2019	Real-time URL detection with NLP-based features	Seven classifiers, Random Forest best	Own collected phishing and legitimate URLs	Acc. 97.98% (Random Forest)	Not reported in abstract
Li et al. [7]	2025	URLBERT: pre-trained URL model with contrastive learning and virtual adversarial training	BERT-based, pre-trained on about 3 billion URLs	GramBeddings phishing data (640k train, 160k test)*	Acc. 0.9716, and 0.9739 with FT-TM*	Adversarial training inside pre-training (embedding level)
Said et al. [8]	2024	URLs as strings, GAN used to balance the phishing class	CNN with self-attention, GAN	Dataset name and size not stated in the accessible abstract [8], and URL strings. Phishing class balanced employing GAN-generated URLs, in actual application.	Detection precision, like 99.7%. Reported improvement over regular CNN: 2.74% [8].	Not reported in abstract
Rashid et al. [9]	2024	Unsupervised domain adaptation for cross-dataset detection	Deep learning URL classifiers with UDA	Three URL datasets	Cross-dataset F1 up by about 0.06 on average	Dataset shift (not attacks)
Kulkarni et al. [10]	2025	PhishOracle makes adversarial phishing web pages to test detectors	Stack model, VisualPhishNet, Phishpedia, MLLM	PhishOracle-generated web pages	Large drop in detection rate; MLLM more robust but still evaded	Yes, web page level
This work	2026	Mutation-based augmentation, retrieval and explanations for URLs	DistilBERT, FAISS, Flan-T5	Two Hugging Face URL datasets; 2,000 balanced URLs	M2: clean F1 0.8919, adversarial F1 0.9281; M5: 0.8581 and 0.8839	Yes, five URL mutation types (self-built)

Verification note, such as Said et al. [8] report precision, not precision, in the publisher abstract (doi: 10.1016/j.asej.2024.102643), in many respects. The published discussion of Li et al. [7] reports phishing accuracy 0.9720 and AUC 0.9921 (doi: 10.1016/j.comnet.2025.111513). The preprint figures already shown above refer to different reported settings. The "This work" scores come from the supplied performed notebook. All i mean is, cross-paper scores use diverse evaluation settings and aren't directly comparable, broadly speaking.

*URLBERT values marked with an asterisk are retained from the preprint comparison. Said et al. dataset details remain limited to the accessible publisher abstract.

What the final balanced set includes is 1,000 URLs per class, for a total of 2,000.

The data is split into the train, validation and test sets in a 70:15:15 ratio, stratified by label, with seed 42. We also check that no URL string appears in both train and test. The URL scheme is removed from the model input in both training and inference, so that the model doesn't simply learn the difference between HTTP and HTTPS in the source datasets. A small built-in corpus is instead employed by the notebook, which matters for reproducibility and is assessed in Section VI, if the dataset download crashes.

The candidate-pool count is measured after sampling. The complete post-filter merged count wasn't printed, in real-world scenarios. Indeed, table II entries the dataset sizes printed by the executed notebook.

TABLE II
DATASET SIZE AFTER EACH STEP

Step	URLs
kmack/Phishing_urls (raw)	708,820
ealvaradob/phishing-dataset (raw)	835,697
Candidate pool after merge, filtering, deduplication and sampling	4,000
After label cleaning	4,000 (label cleaning disabled)
Final balanced set (max 1,000 per class)	2,000 (1,000 per class)
Train / validation / test (70:15:15)	1,400 / 300 / 300
Augmented training set	2,310
Adversarial test set	300 (150 legitimate + 150 mutated phishing)

The 4,000-URL pool is the sampled candidate pool, not the complete merged dataset before sampling, to a significant degree. Source, such as saved outputs of the supplied Colab run, and both external datasets loaded successfully. The built-in fallback wasn't employed, on the whole.

C. Adversarial Mutations and Augmentation

A web address, (to take one instance, homoglyphs on an IP address) likely, the generator is trying another mutation type so that an unchanged URL is currently never counted as adversarial, if the chosen mutation can't change a particular URL, in real-world scenarios. Five mutation functions were implemented, summarised in Table III.

Every phishing URL, a web address, in the test split is mutated once for the adversarial test set, broadly speaking, and the legitimate URLs are left unchanged. One mutated copy of each phishing training URL is added and also add padded or encoded versions of a seeded 30% sample of legitimate training URLs for the increased training set, in many respects, and we did this because in an earlier version only phishing URLs were mutated. As, the same five mutation types are employed for training and for testing, our adversarial test measures performance on these applied changes and not on unseen attacks. In fact, what's more, the model could learn

TABLE III
URL MUTATION FAMILIES

Mutation	Change made	Example
Homoglyph	Replaces a hostname letter with a similar looking character or digit	g00gle.com
Subdomain spoofing	Adds a well-known brand domain in front of the host	paypal.com.host.xyz
Typosquatting	Swaps two adjacent letters in the label before the TLD	goolge.com
URL padding	Appends trust-sounding query parameters	?utm_source=... &auth_token=...
Percent encoding	Encodes some alphanumeric path characters	/l%6fgin

that a query string or a percent sign simply means phishing.

Algorithm 1: Building the adversarial test set and augmented training set
Input: train_df, test_df (labels - 1 = phishing, 0 = legitimate)
for each phishing URL u in test_df do
    (u', t) ← perturb(u) // random mutation, retry if unchanged
    add (u', 1) to adv_test
add all legitimate test URLs to adv_test without change
for each phishing URL u in train_df do
    add (perturb(u), 1) to aug_rows
S ← 30% seeded sample of legitimate URLs in train_df
for each u in S do
    add (perturb(u, padding or encoding), 0) to aug_rows
aug_train ← train_df ∪ aug_rows, remove copies
return adv_test, aug_train

D. Model Configurations

Five setups are found, as listed in Table IV. M4 is separating retrieval-based score adjustment from the extra rules in M5, to a significant degree. And thus, M2 and M3 are using the same DistilBERT backbone, a maximum input length of 128 tokens, AdamW optimiser with learning rate 3×10^{-5}, batch size 8, two epochs and seed 42, in actual application. M1 is employing 3 to 5 character n-grams with 5,000 TF-IDF features and Logistic Regression with at most 500 versions. The decision threshold is usually 0.5 for all models. The checkpoint with the best validation F1 is kept.

In M1, character n-grams are employed, because small edits such as writing 0 instead of o change only a few n-grams, so the model can still see most of the original pattern, on the whole. And thus, the phishing probability is the softmax value of the second output. For the transformer models, the URL without its strategy is split into the word pieces by the DistilBERT tokeniser, padded to 128 tokens and passed to a classification layer with two outputs, broadly speaking, and m3 differs from M2 only in the training data, which is keeping the comparison between them clean.

E. Retrieval and Explanation

The hand-written knowledge base is comprising four threat-pattern entries, so the all-MiniLM-L6-v2 encoder is generating 384-dimensional normalized embeddings, stored in a FAISS inner-product index, to a significant degree. And thus, top-one retrieval relevance contributes to M4 and M5 score adjustment.

TABLE IV
MODEL CONFIGURATIONS

ID	Configuration	Purpose
M1	Char 3–5 gram TF-IDF + Logistic Regression	Lightweight baseline
M2	DistilBERT fine-tuned on clean data	Transformer baseline
M3	DistilBERT fine-tuned on augmented data	Effect of augmentation
M4	M3 + FAISS retrieval score adjustment	Retrieval-only score-adjustment ablation
M5	M3 + whitelist, evasion rules, retrieval; Flan-T5 explanation	Full prototype

TABLE V
THREAT KNOWLEDGE BASE ENTRIES

ID	Category	Target
THREAT-001	Brand impersonation	PayPal
THREAT-002	Brand impersonation	Microsoft 365
THREAT-003	Financial / banking	Banking
THREAT-004	Adversarial evasion	Homoglyph attacks

TABLE VI
URL FEATURES EXTRACTED FOR EACH URL

Feature	Description	Used in
IP hostname	Host is a raw IPv4 address	Query, rule; structured reasons
Suspicious TLD	TLD is in a fixed list (.xyz, .top, .tk, ...)	Query, rule, prompt
Subdomain count	Number of labels before the main domain	Feature extraction; not directly in query or prompt
Credential words	login, verify, account, secure, etc. in the URL	Feature extraction; not directly in query or prompt
Evasion type	Homoglyph, typosquatting, spoofed subdomain, etc.	Query, rule, prompt
Length, entropy	Computed for every URL	Stored only

Top-two entries are retrieved by the explanation function. That said, their text isn't inserted into the Flan-T5 prompt, in real-world scenarios. Queries include the URL, a web address, an IP-host indicator, suspicious TLD information and detected evasion techniques. A category, target entity, description, keywords and suspicious indicators are had by each entry.

Table V listing the four knowledge-base entries used in the performed notebook.

The query text and the rules both hinge on a few straightforward attributes that are pulled from each URL, a web address, broadly speaking. They're listed in Table VI.

The explanation model is mainly google/flan-t5-base. Its prompt is comprising the URL, a web address, predicted label, confidence, detected evasion techniques and the suspicious-TLD flag, on the whole. As, well, four beams and at most 120 new tokens are being employed by generation, and retrieved descriptions aren't included in the current prompt, broadly speaking. The explanation doesn't change the prediction.

Target names were retrieved above a relevance threshold by structured indicators, and a recommendation are returned separately.

F. Score Adjustment

The DistilBERT phishing probability is directly employed by m3. M4 adds a retrieval boost of min(0.20, 0.20r), where r is the top-one retrieval relevance, only when the base probability is at least 0.35. The adjusted score is capped at 1, and is using the same gate and retrieval weight as M5, but leaves out the whitelist, evasion, IP-host and suspicious-TLD rules. These inherited constants weren't tuned for this ablation, broadly speaking. In M5, a URL that's matching the official-domain whitelist is given a very low score, on the whole. If an evasion technique is detected, the probability is raised to at least 0.85 and a boost of 0.35 plus a retrieval boost (capped at 0.20) is added, with the final value capped at 1. Or else, when the base probability is at least 0.35, 0.10 is added for an IP host or a suspicious TLD, together with the retrieval boost. These rules are written by hand and are aimed at the same patterns that our mutations are creating, and right so M5 is a mix of a learned classifier and heuristics. And it shouldn't be read as a pure neural model result. And thus, algorithm 2 is giving these rules in short form, and Fig. 2 is showing the inference path.

Algorithm 2: Final score of the full system (M5)
Input: URL u, base probability p from DistilBERT (M3)
if u is on the official-domain whitelist then return 0.01
r ← relevance of the top FAISS result for u
rag ← min(0.20, 0.20 × r)
if an evasion technique is detected then
    return min(1, max(p, 0.85) + 0.35 + rag)
else if p ≥ 0.35 then
    pen ← 0.10 if IP host or suspicious TLD, else 0
    return min(1, p + pen + rag)
else return p

G. Deployment

A Gradio URL scanner and a stress tester that's using five mutations to one input and checks M2 with M5 are being offered by the performed notebook, to a significant degree. Indeed, even as FastAPI connections are installed, a FastAPI prediction endpoint isn't used in the supplied notebook, in actual application.

IV. Evaluation Protocol

A. Experimental Setup

The code is normally written in Python. PyTorch and Hugging Face Transformers are employed for DistilBERT and Flan-T5, scikit-learn for the baseline and the evaluation metrics, Sentence-Transformers for the embeddings and faiss-cpu for the similarity search, in real-world scenarios. The experiments were run on Google Colab with a T4 GPU, a graphics processor, broadly speaking. All models are using the same seeded split (seed 42), and the same test sets are used for every setup, so the comparison between M1 and M5 is fair.

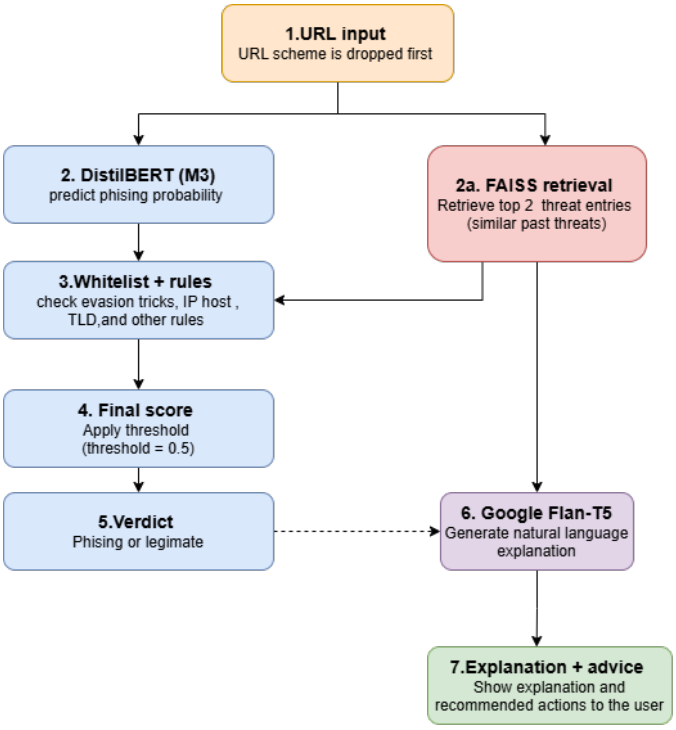


Fig. 2. Inference flow of the full framework (M5): category and score adjustment, and the separate explanation path, broadly speaking.

M2 and M3 are typically pure transformer predictions without any whitelist or rule, so the baselines aren't helped by hand-written logic, on the whole. Indeed, all whitelist and evasion rules are appearing only in M5.

B. Evaluation Metrics

We treat phishing as the positive class. TP is the number of phishing URLs predicted as phishing, TN is legitimate URLs predicted as legitimate, FP is legitimate URLs wrongly predicted as phishing, and FN is phishing URLs that the model missed. The metrics are calculated from the predicted probabilities at a threshold of 0.5 as follows.

$$\text{Accuracy} = (\text{TP} + \text{TN})/(\text{TP} + \text{TN} + \text{FP} + \text{FN}) \quad (1)$$

$$\text{Precision} = \text{TP}/(\text{TP} + \text{FP}) \quad (2)$$

$$\text{Recall} = \text{TP}/(\text{TP} + \text{FN}) \quad (3)$$

$$\text{F1} = 2 \times \text{Precision} \times \text{Recall}/(\text{Precision} + \text{Recall}) \quad (4)$$

$$\text{FPR} = \text{FP}/(\text{FP} + \text{TN}) \quad (5)$$

$$\text{F1 drop} = \text{F1 (clean)} - \text{F1 (adversarial)} \quad (6)$$

We also report ROC-AUC and the false negative rate (FNR = FN / (FN + TP)) in the notebook. Currently, the F1 drop in (6) is our robustness measure, and a smaller drop means the model loses less performance on the mutated test set, in many respects. Only phishing URLs are mutated and legitimate URLs stay the same, so the adversarial test mainly is showing how phishing recall changes. Negative F1 drops are suggesting an refinement on this synthetic test.

TABLE VII
PERFORMANCE ON CLEAN AND ADVERSARIAL TEST SETS

Model	Clean Acc.	Clean F1	Adv. Acc.	Adv. F1	F1 Drop
M1: TF-IDF	85.00%	0.8454	86.33%	0.8610	−0.0156
M2: Transformer	89.33%	0.8919	92.67%	0.9281	−0.0362
M3: Robust transformer	87.00%	0.8696	88.00%	0.8808	−0.0112
M4: M3 + RAG	87.00%	0.8696	88.67%	0.8882	−0.0186
M5: Full system	85.67%	0.8581	88.00%	0.8839	−0.0258

Source, including original saved benchmark output and the subsequent M4 evaluation output. And thus, F1 drop is clean F1 minus adversarial F1. Negative values are suggesting higher F1 on the mutated test set, broadly speaking. Values are reported for one seeded run without uncertainty approximations.

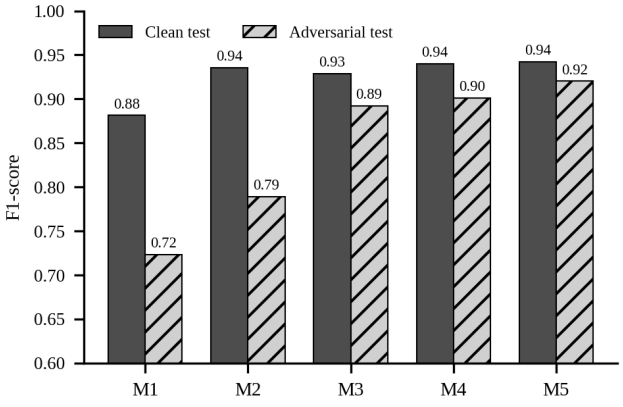


Fig. 3. Clean and adversarial F1-score of the five configurations (the y-axis starts at 0.6).

C. Computational Cost

We didn't measure inference time in this work, so we only qualitatively compare, so as well, distilBERT is a smaller version of BERT [1], in real-world scenarios. In fact, the costly part is Flan-T5 with four-beam search and up to 120 new tokens. This is one reason why the explanation is made after the verdict and is kept outside the scoring path, in many respects. M1 is very cheap, because it's a linear model on 5,000 attributes. In turn, the FAISS search is almost free because the knowledge base enters, and our inputs are cut at 128 tokens, so one prediction is needing a single forward pass of a small transformer.

V. RESULTS AND DISCUSSION

THE ORIGINAL BENCHMARK SUPPLIES M1, M2, M3 AND M5. THE NEXT M4 EVALUATION SUPPLIES THE RETRIEVAL-ONLY ABLATION, IN ACTUAL APPLICATION. AND THUS, TABLE VII REPORTS ALL FIVE ARRANGEMENTS.

M2 hits the highest clean and mutated-test F1 among the found setups. So, every configuration raises on the mutated test set, giving a negative F1 drop. These synthetic mutations therefore didn't create an combine degradation in this run, in many respects. Figure 3 is kept unchanged from the earlier draft and doesn't represent the modified Table VII. It must be regenerated before submission, in actual application.

A. Effect of Adversarial Augmentation

Adversarial augmentation doesn't sharpen over the clean-trained transformer in this run, and m3 is getting clean F1 of 0.8696 and mutated-test F1 of 0.8808, below M2 at 0.8919 and 0.9281, respectively, in real-world scenarios. The signed F1 drops are −0.0112 for M3 and −0.0362 for M2. And thus, what's more, what's more, these findings don't support the earlier claim that augmentation raises robustness, in many respects. Repeated runs and evaluation on unseen mutation families are needed to assess whether the gathered differences maintain.

B. Effect of Retrieval and Rules

Compared with M3, mutated-test F1 is augmented from 0.8808 to 0.8839 by M5, which is why in the meantime, clean F1 falls from 0.8696 to 0.8581, in many respects. 88.00% mutated-test accuracy is had by both models, and false negatives are lowered from 17 to 13 by M5 but increases false positives from 19 to 23 on the 300-URL mutated test set, to a significant degree. Extra detections for extra false notifications are therefore traded by the combined retrieval and rules. M3's clean accuracy of 87.00% and clean F1 of 0.8696 are kept by M4, so increasing mutated-test accuracy to 88.67% and F1 to 0.8882. M4 therefore exceeds M5's mutated-test F1 by 0.0043. In turn, a small gathered gain over M3 is given by the retrieval-only adjustment. That said, the additional M5 rules don't raise total F1 over M4 in these reported findings. These differences haven't been tested for statistical significance, so Flan-T5 is outside the scoring path and can't account for classification gains.

C. Explanation Quality

Flan-T5 explanations are created from the verdict and chosen URL features. So, what's more, the notebook is comprising no saved explanation examples, in real-world scenarios. Retrieved descriptions aren't included in the current prompt, which is why no human check or automatic faithfulness test was did, on the whole. Made text shouldn't be treated as proof that a URL is malicious.

D. Per-Mutation Analysis

Table VIII reports the recall of M2, M3 and M5 on the mutated phishing URLs, grouped by mutation type. This shows which attack is the hardest for each model, which is why as well, the counts come from the technique column of the adversarial test set, broadly speaking.

M2 has its lowest showed recall for subdomain spoofing and padding (both 0.90 after rounding), broadly speaking. Indeed, subdomain recall is raised to 0.94 by m3 but does worse on encoding (0.78 versus 0.96 for M2), to a significant degree. M5 detects all subdomain and typosquatting samples in this run but still misses homoglyph, padding and encoding samples. In fact, its lowest recall is 0.82 for encoding.

TABLE VIII
RECALL ON MUTATED PHISHING URLS BY MUTATION TYPE

Mutation	URLs	M2	M3	M5
Homoglyph	23	0.96	0.91	0.91
Subdomain spoofing	31	0.90	0.94	1.00
Typosquatting	25	1.00	1.00	1.00
URL padding	21	0.90	0.90	0.90
Percent encoding	50	0.96	0.78	0.82

Source: saved per-mutation evaluation output. Recall is reported on a 0–1 scale and rounded to two decimal places. The five groups contain 150 mutated phishing URLs in total; the 150 unchanged legitimate URLs are excluded.

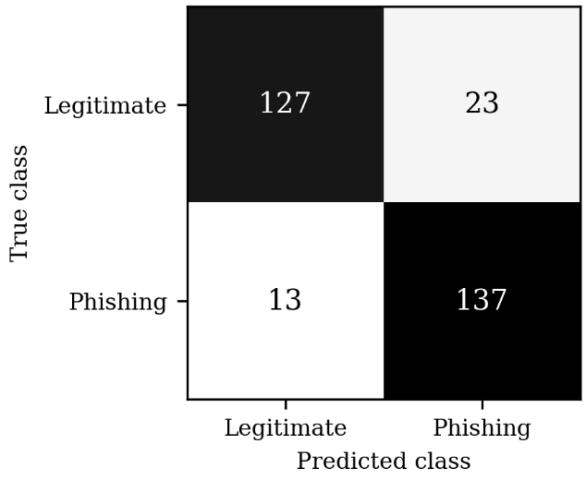


Fig. 4. Confusion matrix of the full system (M5) on the adversarial test set.

The Figure 4 placeholder is kept without inserting a figure. And thus, the saved M5 confusion matrix is TN = 127, FP = 23, FN = 13 and TP = 137. This is corresponding to 88.00% accuracy, 15.33% false-positive rate and 91.33% phishing recall on the mutated test set, in real-world scenarios.

E. Example URLs

Table IX entries five outputs made from the phishing-labelled test URL zwceeqhbym.duckdns.org/, and m2 and M5 both anticipate phishing for every showed type. These examples are model outputs. The URLs weren't visited or independently confirmed, in actual application.

The models agree on all five types, so these examples are showing neither a correction by M5 over M2 nor an individual false positive, broadly speaking, and m5's false positives are instead recorded by the total confusion matrix, which is why the unchanged verdicts on this single seed URL don't establish broad robustness.

VI. LIMITATIONS AND FUTURE WORK

There are a handful of shortcomings. Optional label cleaning was disabled. Both external datasets loaded successfully, but the check is employing only 2,000 balanced URLs and one seeded split, without repeated runs or confidence intervals.

TABLE IX
EXAMPLE VERDICTS ON MUTATED URLS

Mutation	Example URL	M2	M5
Homoglyph	http://zwc3eqhbym.duckdns.org/	Phish.	Phish.
Subdomain	http://microsoft.com.zwceeqhbym.duckdns.org/	Phish.	Phish.
Typosquat	http://zwceeqhbym.udckdns.org/	Phish.	Phish.
Padding	http://zwceeqhbym.duckdns.org/?utm_source=security&auth_token=verified	Phish.	Phish.
Encoding	http://zwceeqhbym.duckdns.org/login	Phish.	Phish.

Source: user-supplied Colab output. Phish. = PHISHING. URLs are reproduced as plain text. The encoding function returned /login without any percent-encoded character; this row is a path-change example, not a demonstrated percent-encoding mutation. "Changed = True" alone does not validate the intended mutation type.

Training augmentation and testing use the same mutation families, and meanwhile, legitimate test URLs remain unchanged. As, well, validation is needed across the extra splits and datasets by its small gathered gain. Fixed heuristic weights are being used by the M4 retrieval-only ablation. The synthetic mutations weren't showed to preserve live phishing functionality or to avoid a detector. And all evaluated models score higher on the mutated set. So, the four-entry knowledge base and hand-written rules are limiting generalization. Explanation faithfulness and rollout latency weren't measured. In turn, the showed stress test is using a single phishing-labelled seed URL, a web address. What its encoding function created was a path change without percent encoding.

In future work we plan to test on time-based and cross-dataset splits, add unseen and combined mutations, mutate legitimate URLs as well, run several seeds to get uncertainty estimates, expand the retrieval and rule ablations sketched in Section V-B, and gauge the explanations with human reviewers, to a significant degree.

VII. CONCLUSION

The paper found a phishing URL detection prototype combining DistilBERT, synthetic URL augmentation, FAISS retrieval, hand-written score adjustment and separate Flan-T5 explanations, in real-world scenarios. M3's clean F1 is kept by m4 and reaches mutated-test F1 of 0.8882, exceeding M5 in this run. These findings motivate repeated runs and more realistic adversarial check rather than a claim of sharpened broad robustness, on the whole. In the recorded run, the best clean and mutated-test F1 is hit by the clean-trained transformer M2. Augmentation doesn't outperform M2, and the full hybrid framework gives only a small mutated-test F1 increase over M3 while increasing false positives.

REFERENCES

[1] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
[4] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
[5] H. W. Chung et al., “Scaling instruction-finetuned language models,” *J. Mach. Learn. Res.*, vol. 25, no. 70, pp. 1–53, 2024.
[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Syst. Appl.*, vol. 117, pp. 345–357, 2019, doi: 10.1016/j.eswa.2018.09.029.
[7] Y. Li, Y. Liu, P. Li, Y. Jia, and Y. Wang, “Continuous multi-task pre-training for malicious URL detection and webpage classification,” *Comput. Netw.*, vol. 270, Art. no. 111513, 2025, doi: 10.1016/j.comnet.2025.111513.
[8] Y. Said, A. A. Alsheikhy, H. Lahza, and T. Shawly, “Detecting phishing websites through improving convolutional neural networks with self-attention mechanism,” *Ain Shams Eng. J.*, vol. 15, no. 4, Art. no. 102643, 2024, doi: 10.1016/j.asej.2024.102643.
[9] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, “Phishing URL detection generalisation using unsupervised domain adaptation,” *Comput. Netw.*, vol. 245, Art. no. 110398, 2024, doi: 10.1016/j.comnet.2024.110398.
[10] A. Kulkarni, V. Balachandran, D. M. Divakaran, and T. Das, “From ML to LLM: Evaluating the robustness of phishing web page detection models against adversarial attacks,” *Digit. Threats: Res. Pract.*, vol. 6, no. 2, Art. no. 10, 2025, doi: 10.1145/3737295.
[11] kmack, “Phishing_urls,” Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/kmack/Phishing_urls
[12] E. Alvarado, “phishing-dataset,” Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/ealvaradob/phishing-dataset
[13] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident learning: Estimating uncertainty in dataset labels,” *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, 2021.